DAY 23

SCHEMA REGISTRY, COMPATIBILITY CHECKS, AND CONTRACT SAFETY

Prevent breaking changes. | Protect every message. | Keep systems safe.

THE GOAL

Evolve message schemas safely across producers and consumers without breaking the system.

WHY SCHEMA GOVERNANCE MATTERS

- Producers & consumers deploy independently.
- Unchecked schema changes break consumers at runtime.
- A schema registry centralizes contracts & compatibility rules.

HOW IT WORKS (HIGH LEVEL)

PRODUCER → SCHEMA REGISTRY → CONSUMER

1 Publish event with schema version

2 Validate schema & compatibility

3 Resolve schema & read message safely

VERSIONED CONTRACTS

- Each subject (topic/event type) has ordered schema versions.
- Producers publish with a schema version identifier.
- Consumers resolve payload shape using registry metadata.

EXAMPLE: USER.CREATED

v1
```
{
 id: string,
 name: string
}
```

v2
```
{
 id: string,
 name: string,
 email: string (optional)
}
```

COMPATIBILITY CHECKS (BACKWARD)

- New schema can read old data safely.
- Do not remove or type-change required fields.
- Adding optional fields is usually safe.

COMPATIBLE
```
{
 id: string,
 name: string,
 email: string (optional)
}
```
Only added an optional field.

INCOMPATIBLE
```
{
 userId: string
}
```
Removed 'name' and changed 'id'.

PUBLISH-TIME SAFETY

- Producers validate payload before publish.
- Invalid data is rejected early (fail fast).
- Prevents bad data from entering the stream.

VALID PAYLOAD
```
{
 id: "U_123",
 name: "Deepak",
 email: "deepak@example.com"
}
```
Accepted
Published to topic

INVALID PAYLOAD
```
{
 id: 123,   // wrong type
 name: null, // required
}
```
Rejected
Not published

TRADE-OFFS

Reliability ↑ Operational overhead ↑	Registry improves reliability but adds governance overhead.
Strict checks ↓ Uncontrolled changes ↓	Strict checks slow changes but prevent production incidents.
Policy & Process Culture & Discipline	Clear evolution policy and playbooks are essential.

END-TO-END FLOW

1. **REGISTER INITIAL SCHEMA** — Team registers v1 schema for a subject
2. **PROPOSE NEW SCHEMA** — Team proposes v2 schema changes
3. **COMPATIBILITY CHECK** — Registry validates compatibility rules
4. **PRODUCER VALIDATION** — Payload validated before publishing
5. **SAFE PUBLISH** — Valid payload accepted; invalid rejected

KEY TAKEAWAYS

- Schema compatibility is a runtime safety concern, not just documentation.
- Backward compatibility enables safer rolling upgrades.
- Publish-time validation catches contract violations early.
- Contract evolution should be explicit and reviewed.
- Registry strategy pairs strongly with event-driven & data platform reliability.

DAY-23 SAMPLE: SCHEMA REGISTRY COMPATIBILITY SIMULATOR

- Register schema versions under a subject
- Validate new versions for backward compatibility
- Reject incompatible schema updates
- Validate outgoing messages against schema
- Demonstrate safe producer evolution from v1 to v2

PROJECT STRUCTURE

```
Day-23/
├── README.md
└── schema-registry-sample/
    ├── python/
    │   ├── schema_registry.py
    │   └── demo.py
    ├── java/
    │   ├── SchemaRegistrySystem.java
    │   └── Main.java
```

RUN THE SAMPLE

PYTHON
```
cd schema-registry-sample\python
python demo.py
```

JAVA
```
cd schema-registry-sample\java
javac Main.java SchemaRegistrySystem.java
java Main
```

NEXT IMPROVEMENTS

- Add forward & full compatibility modes
- Add default values & nullable semantics
- Add schema deprecation lifecycle & ownership metadata
- Add CI contract checks for producer PRs
- Integrate with Avro/ Protobuf & real registry APIs

> Good schemas don't just move data.
> They protect agreements.
> Strong contracts build reliable systems.

CONSISTENT DESIGN.
SAFE EVOLUTION.
RELIABLE SYSTEMS.

20 DAYS OF SYSTEM DESIGN CHALLENGE

System Design - Day 23

Schema Registry, Compatibility Checks, and Contract Safety

Prepared from Day-23 notes and sample implementation

Overview from README.md

Day 23: Schema Registry, Compatibility Checks, and Contract Safety

Day 23 focuses on how distributed systems evolve message schemas safely across producers and consumers.

It covers:
- schema registry fundamentals
- versioned contract management
- backward compatibility checks
- producer/consumer upgrade ordering
- contract validation at publish time
- operational trade-offs in schema governance

In short, Day 23 is about preventing breaking message changes from silently corrupting event-driven systems.

Core ideas

Why schema governance matters
- Producers and consumers usually deploy independently.
- Unchecked schema changes can break downstream parsing at runtime.
- A `schema registry` centralizes contracts and compatibility rules.

Versioned contracts
- Each subject (topic/event type) has ordered schema versions.
- Producers publish with a schema version identifier.
- Consumers resolve payload shape using registry metadata.

Compatibility checks
- Backward compatibility means new schema can read old data safely.
- A common safe rule: required fields should not be removed or type-changed.
- Additional optional fields are typically compatible.

Publish-time safety
- Producer should validate payload against registered schema.
- Unknown fields, missing required fields, or wrong types are rejected early.
- Fail-fast behavior prevents invalid data from entering the stream.

Trade-offs
- Registry improves reliability but adds governance overhead.
- Strict checks slow uncontrolled changes but reduce production incidents.
- Teams need clear evolution policy and migration playbooks.

Day-23 sample: Schema Registry Compatibility Simulator

This repository includes a small schema registry sample in both Python and Java.

Functional requirements
- Register schema versions under a subject
- Validate new versions for backward compatibility
- Reject incompatible schema updates
- Validate outgoing messages against schema

- Demonstrate safe producer evolution from v1 to v2

High-level components
- `SchemaField`: field metadata (name, type, required)
- `SchemaRegistry`: version storage and compatibility checks
- `MessageValidator`: payload validation against a version
- `ProducerSimulator`: sends events only when validation passes

Data flow
1. Team registers initial schema version for a subject
2. Team proposes a new schema version
3. Registry runs compatibility checks against latest version
4. Producer validates payload before publish
5. Valid payload is accepted; invalid payload is rejected with a clear error

Project structure

```
Day-23/
README.md
schema-registry-sample/
python/
schema_registry.py
demo.py
java/
SchemaRegistrySystem.java
Main.java
```

Run the sample

Python

```
cd schema-registry-sample\python
python demo.py
```

Java

```
cd schema-registry-sample\java
javac Main.java SchemaRegistrySystem.java
java Main
```

Interview takeaways
- Schema compatibility is a runtime safety concern, not just documentation
- Backward compatibility enables safer rolling upgrades
- Publish-time validation catches contract violations early
- Contract evolution should be explicit and reviewed
- Registry strategy pairs strongly with event-driven and data platform reliability

Next improvements
- Add forward and full compatibility modes
- Add default values and nullable semantics
- Add schema deprecation lifecycle and ownership metadata
- Add CI contract checks for producer pull requests
- Integrate with Avro/Protobuf and real registry APIs

Reference Implementation

Python and Java sample source files

File: schema-registry-sample/python/demo.py

```python
from schema_registry import ProducerSimulator, SchemaField, SchemaRegistry


def main() -> None:
    registry = SchemaRegistry()
    subject = "order-created"

    v1 = registry.register(
        subject,
        [
            SchemaField("order_id", "string", True),
            SchemaField("amount_cents", "int", True),
        ],
    )
    print(f"registered {subject} schema v{v1.version}")

    v2 = registry.register(
        subject,
        [
            SchemaField("order_id", "string", True),
            SchemaField("amount_cents", "int", True),
            SchemaField("currency", "string", False),
        ],
    )
    print(f"registered {subject} schema v{v2.version} (backward compatible)")

    producer = ProducerSimulator(registry, subject)
    print(
        producer.publish(
            2,
            {"order_id": "ord-1001", "amount_cents": 1599, "currency": "USD"},
        )
    )
    print(
        producer.publish(
            1,
            {"order_id": "ord-1002", "amount_cents": 2099},
        )
    )

    try:
        registry.register(
            subject,
            [
                SchemaField("order_id", "string", True),
                # incompatible: removed amount_cents
            ],
        )
    except ValueError as err:
        print(f"incompatible schema rejected: {err}")

    try:
        producer.publish(2, {"order_id": "ord-1003"})
    except ValueError as err:
        print(f"invalid message rejected: {err}")


if __name__ == "__main__":
    main()
```

File: schema-registry-sample/python/schema_registry.py

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Any


@dataclass(frozen=True)
class SchemaField:
    name: str
    field_type: str
    required: bool


@dataclass(frozen=True)
class SchemaVersion:
    version: int
    fields: list[SchemaField]


class SchemaRegistry:
    def __init__(self) -> None:
        self._subjects: dict[str, list[SchemaVersion]] = {}

    def register(self, subject: str, fields: list[SchemaField]) -> SchemaVersion:
        if not subject:
            raise ValueError("subject must be non-empty")
        if not fields:
            raise ValueError("fields must be non-empty")

        versions = self._subjects.setdefault(subject, [])
        if versions:
            previous = versions[-1]
            self._assert_backward_compatible(previous, fields)

        next_version = len(versions) + 1
        schema = SchemaVersion(version=next_version, fields=fields[:])
        versions.append(schema)
        return schema

    def latest(self, subject: str) -> SchemaVersion:
        versions = self._subjects.get(subject)
        if not versions:
            raise ValueError(f"unknown subject: {subject}")
        return versions[-1]

    def get(self, subject: str, version: int) -> SchemaVersion:
        versions = self._subjects.get(subject)
        if not versions or version <= 0 or version > len(versions):
            raise ValueError(f"schema version not found: {subject} v{version}")
        return versions[version - 1]

    @staticmethod
    def _assert_backward_compatible(previous: SchemaVersion, next_fields: list[SchemaField]) -> None:
        next_map = {field.name: field for field in next_fields}
        for old_field in previous.fields:
            new_field = next_map.get(old_field.name)
            if new_field is None:
                raise ValueError(f"incompatible: removed field '{old_field.name}'")
            if new_field.field_type != old_field.field_type:
                raise ValueError(
                    f"incompatible: field '{old_field.name}' type changed "
                    f"{old_field.field_type} -> {new_field.field_type}"
                )
            if old_field.required and not new_field.required:
                raise ValueError(
                    f"incompatible: required field '{old_field.name}' became optional"
                )


class MessageValidator:
    @staticmethod
    def validate(schema: SchemaVersion, payload: dict[str, Any]) -> None:
```

```
        schema_map = {field.name: field for field in schema.fields}

        for field in schema.fields:
            if field.required and field.name not in payload:
                raise ValueError(f"missing required field: {field.name}")

        for key, value in payload.items():
            field = schema_map.get(key)
            if field is None:
                raise ValueError(f"unknown field: {key}")
            if not MessageValidator._matches(field.field_type, value):
                raise ValueError(
                    f"field '{key}' type mismatch: expected {field.field_type}, got {type(value).__name__}"
                )

    @staticmethod
    def _matches(expected_type: str, value: Any) -> bool:
        return (
            (expected_type == "string" and isinstance(value, str))
            or (expected_type == "int" and isinstance(value, int))
            or (expected_type == "bool" and isinstance(value, bool))
        )


class ProducerSimulator:
    def __init__(self, registry: SchemaRegistry, subject: str) -> None:
        self._registry = registry
        self._subject = subject

    def publish(self, version: int, payload: dict[str, Any]) -> str:
        schema = self._registry.get(self._subject, version)
        MessageValidator.validate(schema, payload)
        return f"published {self._subject} v{version}: {payload}"
```

File: schema-registry-sample/java/Main.java

```
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        SchemaRegistrySystem.SchemaRegistry registry = new SchemaRegistrySystem.SchemaRegistry();
        String subject = "order-created";

        SchemaRegistrySystem.SchemaVersion v1 = registry.register(
            subject,
            List.of(
                new SchemaRegistrySystem.SchemaField("order_id", "string", true),
                new SchemaRegistrySystem.SchemaField("amount_cents", "int", true)
            )
        );
        System.out.println("registered " + subject + " schema v" + v1.getVersion());

        SchemaRegistrySystem.SchemaVersion v2 = registry.register(
            subject,
            List.of(
                new SchemaRegistrySystem.SchemaField("order_id", "string", true),
                new SchemaRegistrySystem.SchemaField("amount_cents", "int", true),
                new SchemaRegistrySystem.SchemaField("currency", "string", false)
            )
        );
        System.out.println("registered " + subject + " schema v" + v2.getVersion() + " (backward compatible)");

        SchemaRegistrySystem.ProducerSimulator producer = new SchemaRegistrySystem.ProducerSimulator(registry, s

        Map<String, Object> payloadV2 = new LinkedHashMap<>();
        payloadV2.put("order_id", "ord-1001");
        payloadV2.put("amount_cents", 1599);
        payloadV2.put("currency", "USD");
        System.out.println(producer.publish(2, payloadV2));

        Map<String, Object> payloadV1 = new LinkedHashMap<>();
        payloadV1.put("order_id", "ord-1002");
        payloadV1.put("amount_cents", 2099);
        System.out.println(producer.publish(1, payloadV1));

        try {
            registry.register(
                subject,
                List.of(
                    new SchemaRegistrySystem.SchemaField("order_id", "string", true)
                )
            );
        } catch (IllegalArgumentException err) {
            System.out.println("incompatible schema rejected: " + err.getMessage());
        }

        try {
            Map<String, Object> badPayload = new LinkedHashMap<>();
            badPayload.put("order_id", "ord-1003");
            System.out.println(producer.publish(2, badPayload));
        } catch (IllegalArgumentException err) {
            System.out.println("invalid message rejected: " + err.getMessage());
        }
    }
}
```

File: schema-registry-sample/java/SchemaRegistrySystem.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class SchemaRegistrySystem {
    public static final class SchemaField {
        private final String name;
        private final String fieldType;
        private final boolean required;

        public SchemaField(String name, String fieldType, boolean required) {
            this.name = name;
            this.fieldType = fieldType;
            this.required = required;
        }

        public String getName() {
            return name;
        }

        public String getFieldType() {
            return fieldType;
        }

        public boolean isRequired() {
            return required;
        }
    }

    public static final class SchemaVersion {
        private final int version;
        private final List<SchemaField> fields;

        public SchemaVersion(int version, List<SchemaField> fields) {
            this.version = version;
            this.fields = fields;
        }

        public int getVersion() {
            return version;
        }

        public List<SchemaField> getFields() {
            return fields;
        }
    }

    public static final class SchemaRegistry {
        private final Map<String, List<SchemaVersion>> subjects = new LinkedHashMap<>();

        public SchemaVersion register(String subject, List<SchemaField> fields) {
            if (subject == null || subject.isBlank()) {
                throw new IllegalArgumentException("subject must be non-empty");
            }
            if (fields == null || fields.isEmpty()) {
                throw new IllegalArgumentException("fields must be non-empty");
            }

            List<SchemaVersion> versions = subjects.computeIfAbsent(subject, key -> new ArrayList<>());
            if (!versions.isEmpty()) {
                SchemaVersion previous = versions.get(versions.size() - 1);
                assertBackwardCompatible(previous, fields);
            }

            int nextVersion = versions.size() + 1;
            SchemaVersion schema = new SchemaVersion(nextVersion, new ArrayList<>(fields));
            versions.add(schema);
            return schema;
        }

        public SchemaVersion latest(String subject) {
```

```
        List<SchemaVersion> versions = subjects.get(subject);
        if (versions == null || versions.isEmpty()) {
            throw new IllegalArgumentException("unknown subject: " + subject);
        }
        return versions.get(versions.size() - 1);
    }

    public SchemaVersion get(String subject, int version) {
        List<SchemaVersion> versions = subjects.get(subject);
        if (versions == null || version <= 0 || version > versions.size()) {
            throw new IllegalArgumentException("schema version not found: " + subject + " v" + version);
        }
        return versions.get(version - 1);
    }

    private static void assertBackwardCompatible(SchemaVersion previous, List<SchemaField> nextFields) {
        Map<String, SchemaField> nextMap = new LinkedHashMap<>();
        for (SchemaField field : nextFields) {
            nextMap.put(field.getName(), field);
        }

        for (SchemaField oldField : previous.getFields()) {
            SchemaField newField = nextMap.get(oldField.getName());
            if (newField == null) {
                throw new IllegalArgumentException("incompatible: removed field '" + oldField.getName() + "'
            }
            if (!newField.getFieldType().equals(oldField.getFieldType())) {
                throw new IllegalArgumentException(
                    "incompatible: field '"
                        + oldField.getName()
                        + "' type changed "
                        + oldField.getFieldType()
                        + " -> "
                        + newField.getFieldType()
                );
            }
            if (oldField.isRequired() && !newField.isRequired()) {
                throw new IllegalArgumentException(
                    "incompatible: required field '" + oldField.getName() + "' became optional"
                );
            }
        }
    }
}

public static final class MessageValidator {
    public static void validate(SchemaVersion schema, Map<String, Object> payload) {
        Map<String, SchemaField> schemaMap = new LinkedHashMap<>();
        for (SchemaField field : schema.getFields()) {
            schemaMap.put(field.getName(), field);
            if (field.isRequired() && !payload.containsKey(field.getName())) {
                throw new IllegalArgumentException("missing required field: " + field.getName());
            }
        }

        for (Map.Entry<String, Object> entry : payload.entrySet()) {
            String key = entry.getKey();
            Object value = entry.getValue();
            SchemaField field = schemaMap.get(key);
            if (field == null) {
                throw new IllegalArgumentException("unknown field: " + key);
            }
            if (!matches(field.getFieldType(), value)) {
                throw new IllegalArgumentException(
                    "field '"
                        + key
                        + "' type mismatch: expected "
                        + field.getFieldType()
                        + ", got "
                        + value.getClass().getSimpleName()
                );
            }
        }
```

```
        }

        private static boolean matches(String expectedType, Object value) {
            return ("string".equals(expectedType) && value instanceof String)
                || ("int".equals(expectedType) && value instanceof Integer)
                || ("bool".equals(expectedType) && value instanceof Boolean);
        }
    }

    public static final class ProducerSimulator {
        private final SchemaRegistry registry;
        private final String subject;

        public ProducerSimulator(SchemaRegistry registry, String subject) {
            this.registry = registry;
            this.subject = subject;
        }

        public String publish(int version, Map<String, Object> payload) {
            SchemaVersion schema = registry.get(subject, version);
            MessageValidator.validate(schema, payload);
            return "published " + subject + " v" + version + ": " + payload;
        }
    }
}
```